

CANDY

DIALOGUE ENGINE

Suspend Dialogue

1.0.0

If required, dialogue can be suspended (paused) at any moment: when lines are writing, when commands are being processed, or when the dialogue is idle and waiting for player input.

When dialogue is suspended, the following will happen will:

- Pause any ongoing writing (typewriter effect) of spoken lines.
- Pause portrait animations.
- Pause voice playback (if lines have voice files)
- Put the processing of new lines on hold.
- Pause all media players related to that dialogue instance, including image players.
- Pause AnimationPlayers related to that dialogue instance (those which are used by the §Effect command).
- Pause animations and effects for Backgrounds (if you have a Backgrounds scene in your UI)
- Pause animations and effects for VN busts (if you have a Bust scene in your UI).

Note that animations triggered with the §CS commands (e.g. character movement or animations) won't be paused: these are game events, dialogues only trigger them.

When dialogue is resumed, everything above will resume, with one detail: any media and animations that were not paused by the dialogue suspension will remain paused. For example, if a media was paused by a command before dialogue was suspended, it will remain paused. This ensures that suspending dialogue won't disrupt your dialogue timeline.

Dialogue can be suspended or resumed in any way you want: by pressing a key, clicking a button, or automatically upon specific game events.

For some examples, this can be used for the following:

- If the player just wants to pause the game at any time.
- If the player opens the inventory, the history log, or another menu.
- If the player clicks on Lexicon keywords.

- If a game-event suddenly occurs and momentarily interrupts dialogue.

Setup

Code

By default, dialogue suspension is specific to individual instances of **Candy_Engine.gd**.

Each instance of the script has its own **dialogue_suspended** variable. All you need to do, is toggle it between 'true' or 'false'. When an engine instance processes lines, it checks the value of this variable and, if 'true', will wait for it to switch to 'false' before continuing.

Note that toggling the variable only suspends dialogue processing. In order to suspend media, animations, voices, portraits, backgrounds and VN busts, you'll also need to call the **suspend_dialogue()** function inside a given engine instance.

Here is an example code, for toggling dialogue suspension with the press of a key:

```
func _input(event: InputEvent) -> void:
    if event.is_action_pressed("Pause_Dialogue"):
        candy_ui.dialogue_suspended = not candy_ui.dialogue_suspended
        if candy_ui.dialogue_suspended == true:
            candy_ui.suspend_dialogue()
        elif candy_ui.dialogue_suspended == false:
            candy_ui.resume_dialogue()
```

Notice that the code above toggles **dialogue_suspended** and calls the **suspend_dialogue()** function for a specific engine instance: **candy_ui**.

If you want to suspend multiple engine instances, you could compile them in an array, then use a 'for' loop to iterate through the array and suspend or resume each instance.

Notice also that the code above toggles suspension: it doesn't care whether dialogue is suspended or not, it always switches it to the opposite state.

If you try to suspend multiple engine instances, this could be a problem: if instances are created after some existing instances were suspended, these new instances won't be suspended. When you toggle all instances, those that were suspended will resume, and those that weren't suspended will suspend.

There's an easy solution: use a global variable as an authority on dialogue state. For example, write your code to toggle that global variable, then suspend or resume all engine instances based on the value of that variable, like this:

```
func _input(event: InputEvent) -> void:
    if event.is_action_pressed("Pause_Dialogue"):
        globals.suspend_authority = not globals.suspend_authority  #/ Toggle
        for i in globals.engine_instance:  #/ Array of instances to suspend
            if globals.suspend_authority == true:
                i.dialogue_suspended = true
                i.suspend_dialogue()
            elif globals.suspend_authority == false:
                i.dialogue_suspended = false
                i.resume_dialogue()
```

Note that the example above is specifically for suspending dialogue through a key press, but you can use the same logic for suspending multiple engine instances through different events.

If your game has a set number of engine instances, you can populate the array of instances to suspend manually. If instances can be created dynamically at runtime (such as NPC barks), you just need to write a small code to append new instances to the array when they're created (and don't forget to remove them from the array when they disappear).

Configuration

Candy DE lets you configure which dialogue elements are suspended. By default, all are.

You can exempt some elements through the 'suspend_' variables, in [Candy_Engine.gd](#):

```
#^ Suspend dialogue elements:
#? Set to false for things you don't want to be paused when dialogue_suspended = true.
@export var suspend_portraits: bool = true      #/ Suspend portrait animations
@export var suspend_voice: bool = true          #/ Suspend voice playback for spoken lines

@export var suspend_backgrounds: bool = true    #/ Suspend background animations and effects
@export var suspend_bg_animations: bool = true  #/ Suspend background animations
@export var suspend_bg_effects: bool = true     #/ Suspend background effects

@export var suspend_vn_busts: bool = true       #/ Suspend VN bust animations and effects
@export var suspend_bust_animations: bool = true #/ Suspend VN bust animations
@export var suspend_bust_effects: bool = true   #/ Suspend VN bust effects

@export var suspend_media: bool = true          #/ Suspend all media ($Sound, $Video, $Image, $Effect commands)
@export var suspend_audio: bool = true          #/ Suspend audio media ($Sound command)
@export var suspend_video: bool = true          #/ Suspend video media ($Video command)
@export var suspend_images: bool = true         #/ Suspend image media ($Image command)
@export var suspend_animations: bool = true     #/ Suspend visual effects ($Effect command)
```

```
var suspend_portraits = true      #/ Portrait animations
var suspend_voice = true          #/ Voice playback

var suspend_backgrounds = true    #/ Background animations/effects
var suspend_bg_animations = true  #/ Background animations only
var suspend_bg_effects = true     #/ Background effects only

var suspend_vn_busts = true       #/ VN Bust animations/effects
var suspend_bust_animations = true #/ Bust animations only
var suspend_bust_effects = true   #/ Bust effects only

var suspend_media = true          #/ $Sound, $Video, $Image, $Effect
var suspend_audio = true          #/ $Sound only
var suspend_video = true          #/ $Video only
var suspend_images = true         #/ $Image only
var suspend_animations = true     #/ $Effect only
```

(Note that these variables are exported and can be modified in the Inspector.)

This is useful in situations where you might want background visuals or audio to continue playing even while dialogue is paused.

Of course, you might want to prevent specific media players from being suspended, instead of the current all/none. In that case, simply add these players to the **media_suspend_immune** array. Specifically, you must add their keys from the **media_players_locations** dictionary (also found in **Candy_Engine.gd**).

For VN Busts, note that JML AnimationPlayers will also be suspended if **suspend_vn_busts** and **suspend_bust_effects** are both true. In fact, all AnimationPlayers in the Busts scene will be suspended. However, other JML effects (such as tween animations) won't be suspended. Since any JML effects not using AnimationPlayer nodes need to be custom-coded by yourself, you'll need to write code to suspend them as well, if you want that.

Inactive Dialogue Engines

What happens if **dialogue_suspended** is toggled, or **suspend_dialogue()** is called on an engine instance when it isn't running any dialogue?

Nothing to worry about!

dialogue_suspended will indeed change its value as normal. If a dialogue is started while the value is false, the dialogue will immediately suspend and wait for the value to change to 'true'.

Once the value changes to 'true', it will automatically proceed with the first line. You don't need to restart the dialogue, or to do anything else.

Meanwhile, calling **suspend_dialogue()** while no dialogue is running will have no effect, because it only pauses active media players, backgrounds or vn busts associated to the specific dialogue engine instance.

However, it can happen that media, backgrounds or vn busts remain active even when dialogue isn't running. This can occur if you configure backgrounds or vn busts to persist after dialogue ends (in the **Candy_Engine.gd** settings), if another engine instance shares the same nodes, or if your game code made the nodes play media or animations.

In that case, **suspend_dialogue()** will pause these nodes, but only if it's configured to do so (e.g. it won't pause persistent backgrounds if you remember to set **suspend_backgrounds** to 'false' in the settings).

It's up to you to think carefully about how you configure the dialogue engine settings, to avoid such conflicts. Note that sharing UI elements between engine instances is **not** recommended specifically because it can introduce such complex situations (although there might be legitimate cases to do so, and it can be done safely).

Alternately, wherever your code toggles **dialogue_suspended** or calls **suspend_dialogue()**, you can add logic to exclude engine instances that are not running. **Candy_Engine.gd** has a convenient boolean variable you can use for this purpose: **dialogue_running**. It's 'false' by default, and set to 'true' when dialogue starts (i.e. when **start_dialogue()** is called), then set back to 'false' when dialogue ends (when **end_dialogue()** is called).

It doesn't change otherwise (e.g. it isn't set to 'false' when dialogue is suspended, only when it ends), so it always tells you if a dialogue is active in the given engine instance, even if at the moment the dialogue itself is suspended or idle.

Just check that `dialogue_running` is 'true': if it's 'false', configure your code to not change `dialogue_suspended` and to not call `suspend_dialogue()` in the given instance.

Also, be mindful that `dialogue_running` won't be set to 'false' if dialogue doesn't end properly, i.e. by calling `end_dialogue()` (such as if a bug causes the engine instance to crash or abort dialogue unexpectedly). `end_dialogue()` always prints "**end_dialogue(): Dialogue Ended**" in the output, to let you know it did its job and the dialogue was ended correctly.

Code Customization

The functions for pausing the various dialogue elements are straightforward and therefore easy to modify. They look for specific media player nodes, check whether they're meant to be suspended, and pause them. They also add any nodes they pause to arrays, so that they can later resume only those nodes.

You could easily add extra checks if you need more precise control. For example, you could add a 'var suspend_immune' variable to the scripts of individual media players, and edit the media suspension functions to check the value of that variable for each node before pausing it.

As an example, this could allow you to pause only specific `AnimationPlayer` nodes in Background or VN Bust scenes. You could even go further, and use a dictionary in each media player node to make different rules for each dialogue engine instance.

With that said, we expect the default options will be more than enough for most use-cases, so you probably won't need to modify the code at all. The examples above are just meant to show that the dialogue suspension feature can be made much more powerful without complicated work, if desired.